



Laboratorio 1

Introducción a la terminal

Jesus Humberto Abundis Patiño

27th February 2026



Introducción

Teoría

Terminal y shell

Una **terminal** es una interfaz basada en texto (ventana o emulador de software), que acepta entradas del usuario y muestra datos de salida, la cual nos permite interactuar directamente con el sistema operativo. La entrada del usuario es enviada a un programa que corre dentro de la **terminal**, conocido como **shell** o **intérprete** o **intérprete de línea de comandos**, el cual interpreta dicha entrada como un comando y lo ejecuta dentro de un **ambiente**.

En resumen, la **terminal** provee la ventana donde introducir comandos y ver resultados, mientras que el **shell** es el que ejecuta dichas instrucciones.

Por su parte, el **ambiente** se refiere al conjunto de ajustes, variables, alias y funciones, que definen como operarán los programas que corren dentro del shell, es decir, el contexto en el que correrán nuestras aplicaciones.

Ejemplos de terminales o interfaces de linea de comando (CLI):

- xterm,
- GNOME Terminal,
- Konsole,
- Alacritty,
- Terminal.app,
- iTerm2.

Ejemplos de intérpretes:

- **bash** (Bourne Again SHell),
- sh (Bourne shell),
- zsh (Z shell),
- cmd.exe (Command Prompt),
- PowerShell.

De los intérpretes anteriores, el más ampliamente usado en Linux es **bash**, para el cual mostraremos los comandos fundamentales. En particular, estaremos utilizando una terminal en Ubuntu, la distribución de Linux mas empleada.

Bash shell en Ubuntu

Comencemos observando el sistema de archivos de Ubuntu en el siguiente gráfico.

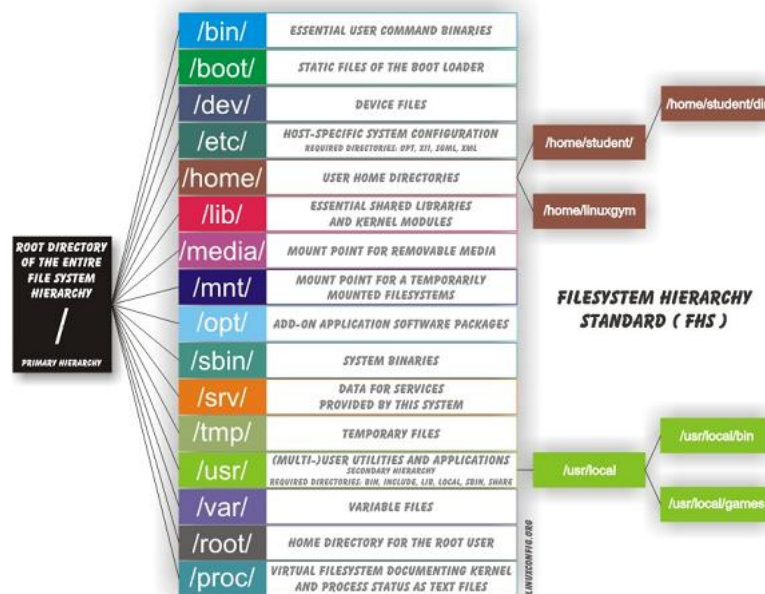


Figure 0.1: Estándar de jerarquía del sistema de archivos (HFS) en Linux. [1]

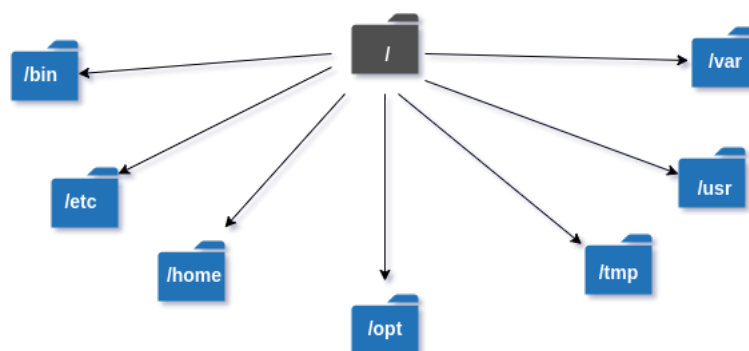


Figure 0.2: Algunos directorios importantes en linux. [2]

A continuación describimos algunos de los comandos más importantes de **bash**.

Lo más simple que podemos hacer con el intérprete es simplemente ejecutar un programa:

- pwd** Este programa imprime nuestra ubicación en el sistema de archivos.
- date** Muestra la fecha y hora.
- whoami** Muestra el usuario.

También podemos darle argumentos a nuestros programas. Los argumentos se separan en bash mediante uno o más espacios en blanco (también pueden ser tabs o nuevas líneas), dónde la primer palabra se referirá al programa y las demás serán los argumentos del programa. **Nota:** Espacios en blanco consecutivos se consideran como un sólo delimitador.

<code>echo hello world</code>	Imprimir argumentos (dos argumentos).
<code>echo hello world</code>	(aún dos argumentos)
<code>echo 'hello world'</code>	(un solo argumento).
<code>echo hello\ world</code>	utilizando el caracter de escape \ (un solo argumento).

Para introducir comillas debemos emplear comillas dobles y a la inversa las comillas dobles se logran mediante comillas simples. Otra alternativa es usar el caracter de escape \.

```
echo "Robert's house"
echo Robert\' house
```

Para obtener ayuda podemos utilizar

```
man date
grep --help
```

Cambiamos de directorio utilizando **direcciones absolutas** y **relativas**. Las **direcciones absolutas** empiezan con una /, mientras que las **relativas** empiezan con el nombre de un directorio presente en nuestra ubicación actual. Además, existen dos símbolos para directorios especiales: '.' se refiere al directorio actual y '..' se refiere al directorio padre. **Nota:** Podemos utilizar la tecla **TAB** para autocompletar.

<code>cd /bin</code>	Cambiar al directorio bin dentro de la raíz.
<code>cd -</code>	Regresa al directorio anterior
<code>cd /</code>	
<code>cd bin</code>	
<code>cd .</code>	
<code>cd ../</code>	
<code>cd bin/../bin/./</code>	¿En qué directorio nos quedamos?

Una localidad importante es el **directorio de usuario**, también conocido como el directorio **home**. Existe el símbolo ~ para referirnos a esta ubicación:

<code>cd ~</code>	
<code>cd ~/..</code>	Un directorio arriba del de usuario
<code>cd</code>	Sin argumentos nos lleva al directorio de usuario también

Otros comandos importantes son

<code>mkdir MyDir</code>	Crea directorio <code>MyDir</code> en la ubicación actual,
<code>mkdir /bin/OtroDir</code>	Crea directorio <code>OtroDir</code> al interior del directorio <code>bin</code> ,
<code>rmdir /bin/OtroDir</code>	Elimina directorio <code>OtroDir</code> al interior del directorio <code>bin</code> ,
<code>touch MyDir/file1</code>	Crea archivo <code>file1</code> dentro de <code>MyDir</code> . Si el archivo existiese solo actualizaría su hora de creación.
<code>cd MyDir</code>	
<code>touch file2 file3</code>	
<code>ls</code>	Lista los archivos y directorios dentro de la ruta actual,
<code>ls /bin</code>	Lista los archivos y directorios en la ruta deseada.

Para copiar, mover y eliminar archivos tenemos

```
cp archivo1 archivo2 archivo3 Directorio
mv archivo1 archivo2 Directorio
rm archivo2 archivo3
```

En Linux contamos con la **redirección de salida** mediante los operadores `>` y `>>`. Con estos, enviamos el **standard output** (`stdout` o salida estándar) de un comando hacia un archivo, en vez de mostrarlo en pantalla. Por Ej.,

<code>ls > lista.txt</code>	El comando <code>ls</code> , retorna los nombres de los archivos y directorios a stdout y estos son redireccionados (almacenados) en <code>lista.txt</code> . Si el archivo <code>lista.txt</code> ya existiese, su contenido sería reemplazado.
--------------------------------	---

<code>echo "línea A" > lista.txt</code>	
<code>ls > lista.txt</code>	
<code>echo "línea A" >> lista.txt</code>	De esta manera, si el archivo no existe, se crea y se le agrega el resultado del comando. Si el archivo existe, agregamos el resultado a la parte inferior de este en vez de reemplazarlo.

```
echo "línea B\nlínea C" >> lista.txt
```

¿Qué ocurre cuando introducimos una instrucción en la terminal?

Cada que ejecutamos un programa en Linux, digamos el programa `cd`, este busca un ejecutable con ese nombre en alguno de los directorios listados en la variable de ambiente `PATH`. Busca en el primer directorio de esta lista, si no lo encuentra continua con el siguiente y una vez encontrado lo ejecuta. En otras palabras, si dos ejecutables tienen el mismo nombre que el programa que escribí en la terminal, se ejecutará el que se encuentre primero en la lista de `PATH`.

Para saber la ubicación del archivo ejecutable de un programa, utilizamos el comando `which` como a continuación;

```
which date
which which
```

which -a date Con esta opción obtenemos todos los ejecutables con el nombre dado en caso de haber más de uno.

Las **variables de ambiente** son otro elemento importante en el intérprete; estas son utilizadas por aplicaciones y el shell para cambiar su comportamiento, para localizar los archivos ejecutables (programas), y compartir información entre procesos. Estas son globales para el intérprete dónde se definen y se heredan automáticamente a todo proceso hijo o programa ejecutado desde el shell. Por convención, estas se escriben en mayúsculas para distinguirlas de variables locales del shell.

Algunas variables de ambiente importantes en Linux:

PATH	Lista de directorios donde el intérprete busca archivos ejecutables (los programas)
SHELL	Ruta del intérprete por default
HOME	Ruta del directorio de usuario
USER	Nombre de usuario activo
EDITOR	Editor por default

Para ver todas las variables de ambiente:

```
env  
printenv
```

Otros comando útiles para estas variables:

export	Por Ej., export COMPILER=gcc , crear variable de ambiente COMPILER ,
unset	Por Ej., unset COMPILER , eliminar variable de entorno COMPILER .

Para ver el valor de una variable de ambiente podemos utilizar la expansión de parámetros mediante **\$VAR**, donde **VAR** es una variable de ambiente o local del shell.

echo \$PATH **bash** expande el valor de la variable **PATH** y posteriormente se ejecuta la instrucción.

Bibliography

- [1] *How to understand the Ubuntu file system layout?* URL: <https://askubuntu.com/questions/138547/how-to-understand-the-ubuntu-file-system-layout>.
- [2] *Linux Directory Structure.* URL: <https://www.geeksforgeeks.org/linux-unix/linux-directory-structure/>.